

Sprog og oversættelse

Lexical analysis: Regular expressions and NFAs

Part 4: Nondeterministic Finite Automata

Hans Hüttel

(using content by Cosmin Oancea)

Creative Commons License
Attribution Non Commercial 4.0 International
(CC-BY-NC-4.0)

From regular expressions to lexers



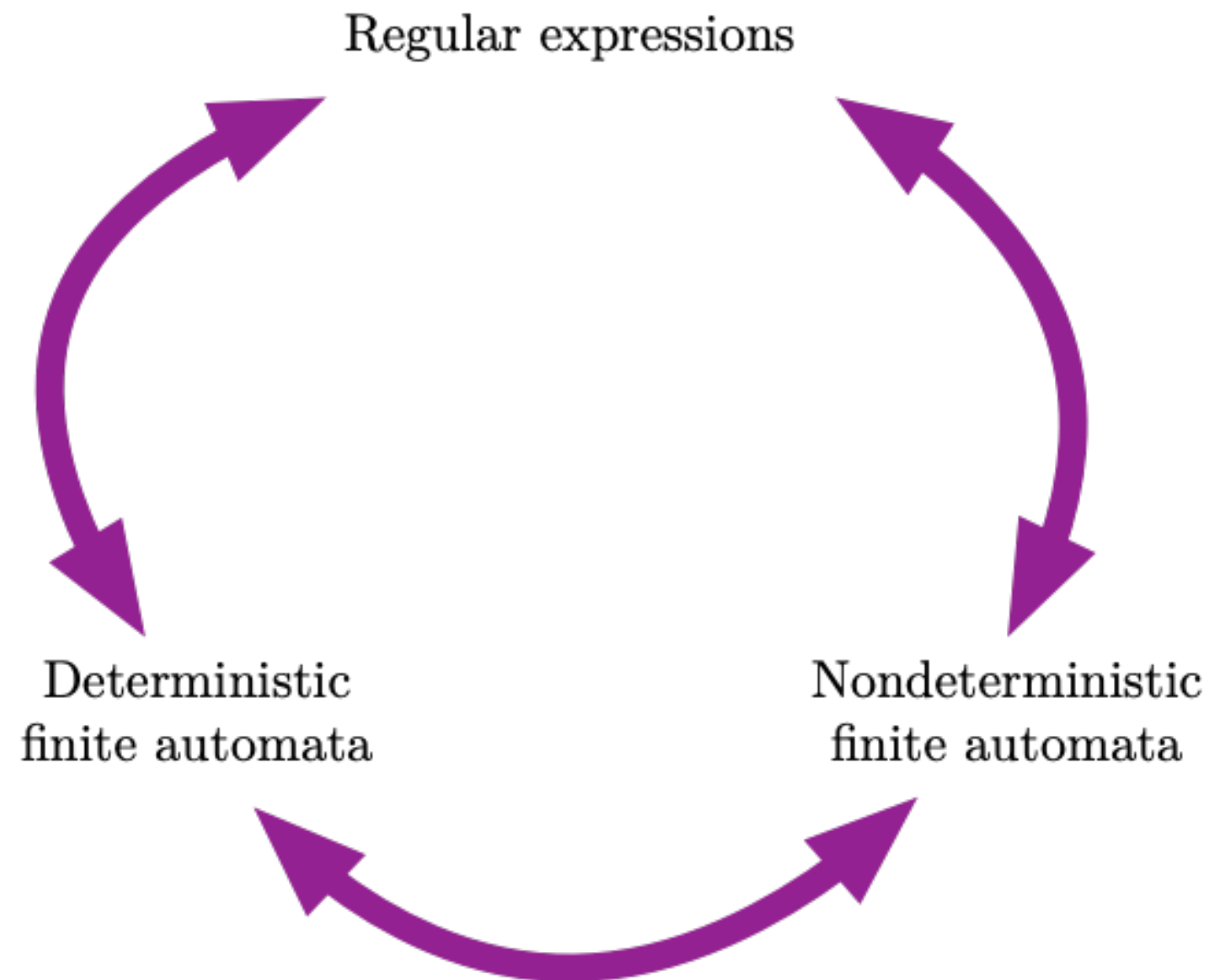
From any regular expression we can build a lexical analyser.

There is an algorithm for this!

Lexer generators such as `lex`, `flex` and `re2c` make use of variations of this algorithm.

The theorem behind it all

A result, known as **Kleene's theorem**, tells us that regular expressions have the same expressive power as two different machine models.



Nondeterministic Finite Automata (NFA)

The notion of NFA is a stepping stone when we want to obtain a lexer for a regular expression.

An NFA is **nondeterministic**: it makes guesses!

(But any NFA can be transformed to a **deterministic** FA (DFA), which does not make guesses. This DFA is an actual lexer! More about that next time.)

Nondeterministic Finite Automata (NFA)

An NFA is a machine with a **finite** set of

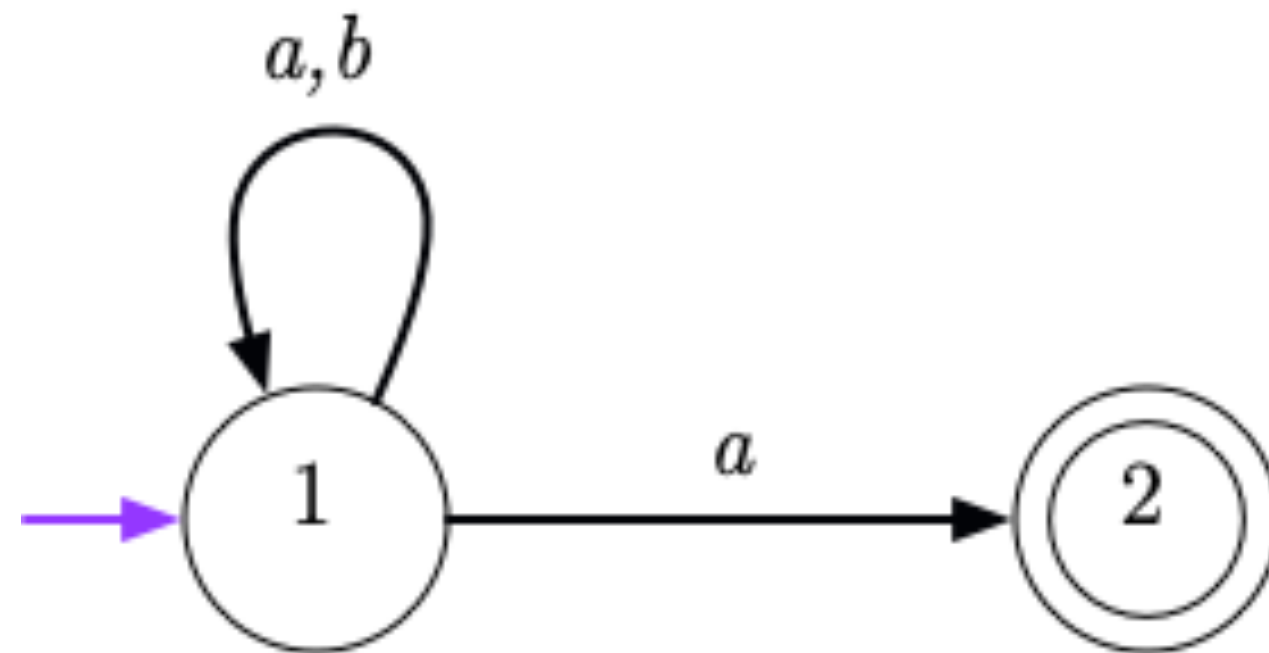
- states ²: initial, final, intermediate,
- transitions between states labelled with a char $c \in \Sigma$, or with ϵ

An NFA makes guesses as it reads its input.

For any state there can be more than one outgoing transition labelled with the same character. There can also be state changes not caused by reading a character: these are ϵ -transitions.

²Dansk: *tilstande* – **det hedder aldrig ”stadier”!**

A simple NFA



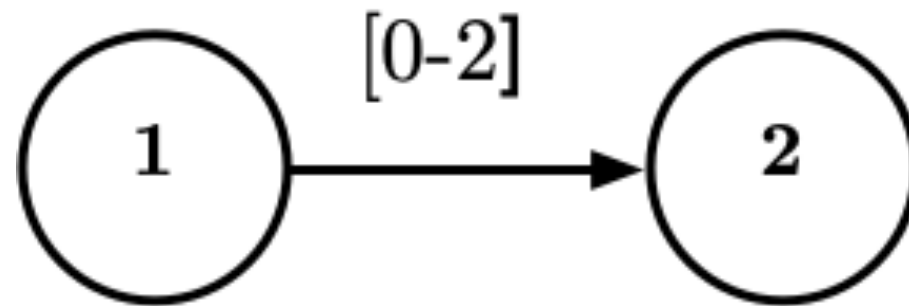
This is an NFA that accepts every string that is in the language described by the regular expression

$$(a|b) * a$$

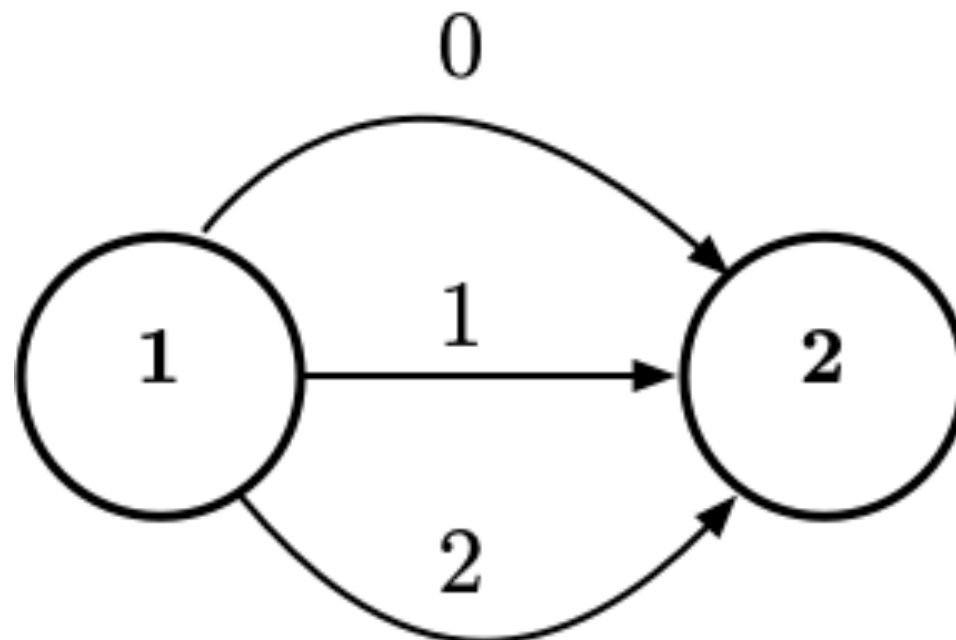
State 1 is the initial state, state 2 is a final state.

A convenient shorthand

We often label transitions with ranges, such that we can draw



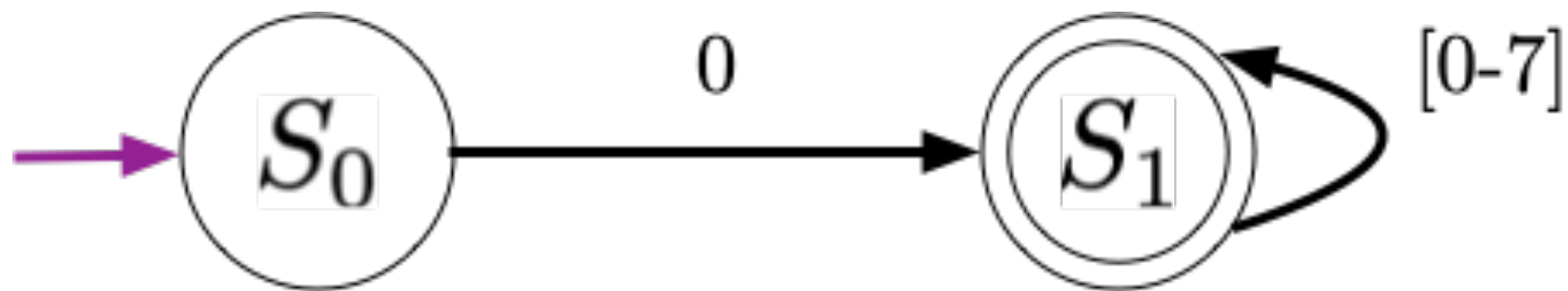
instead of



An NFA for octal numbers

A regular expression for the language of octal numbers is

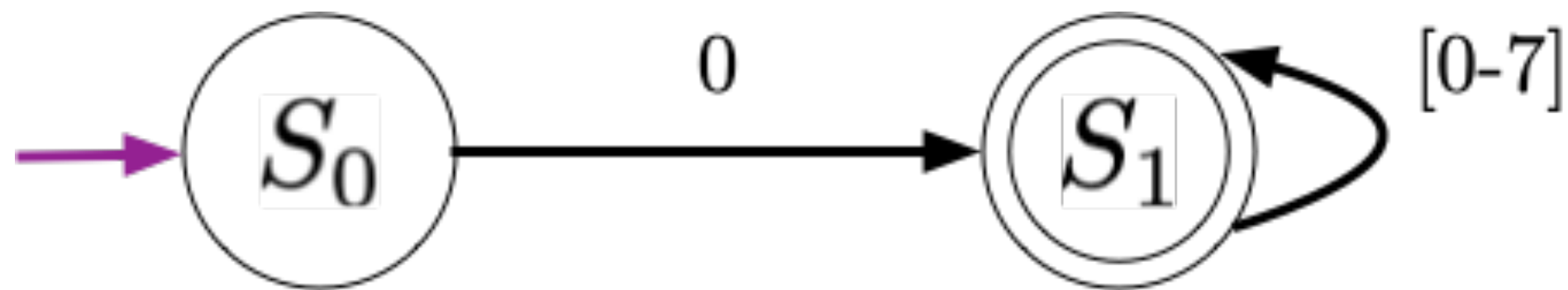
$$0[0 - 7]^*$$



An NFA for octal numbers

A regular expression for the language of octal numbers is

$$0[0 - 7]^*$$

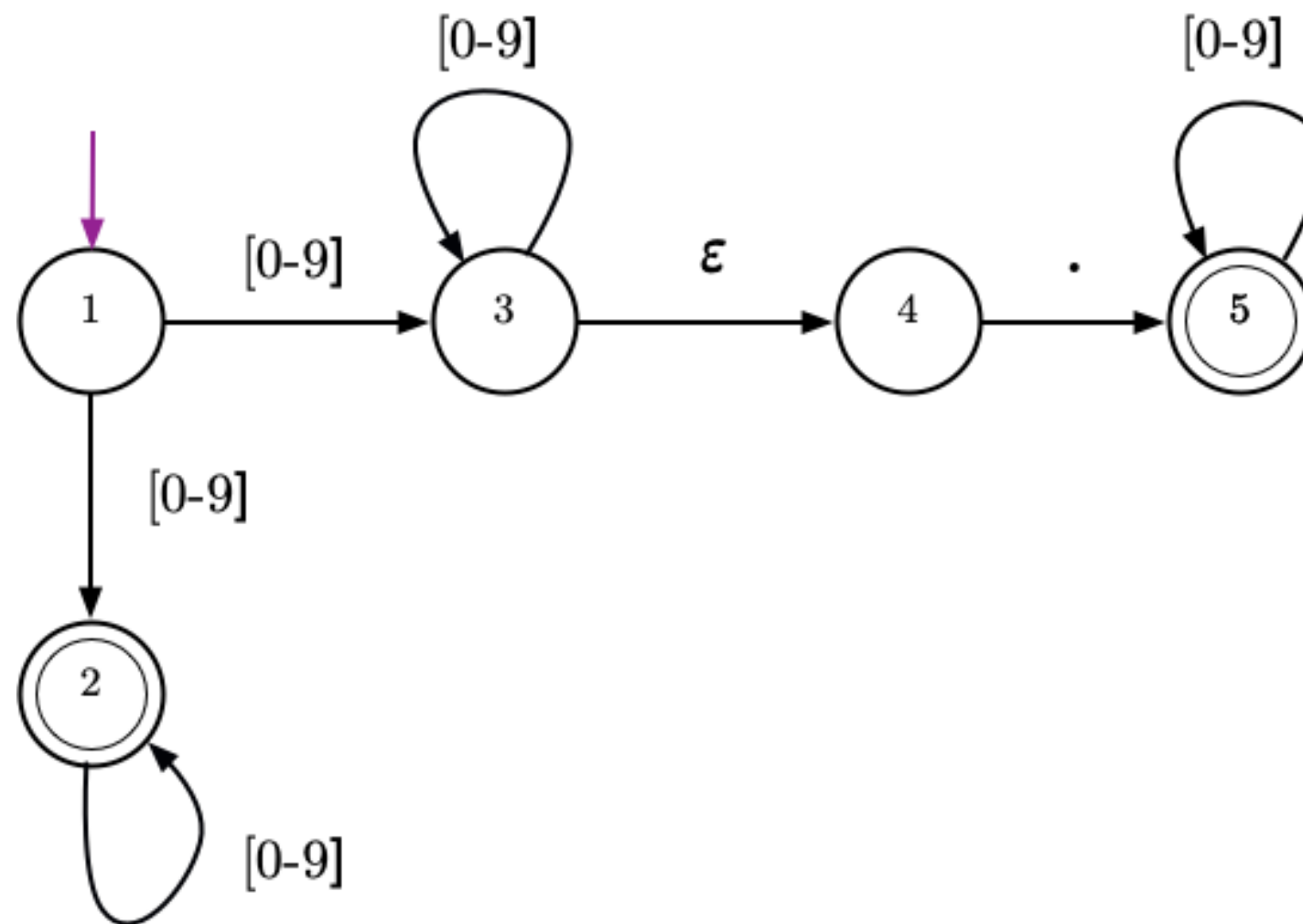


- Start in state S_0
 - if input is 0 go to state S_1
 - otherwise the machine fails!
- In state S_1 :
 - if input in $0 \dots 7$ stay there
 - otherwise the machine fails!
 - If the end of input was reached, an octal number was identified!

An NFA for decimal numbers

Decimal numbers can be described by the regular expression

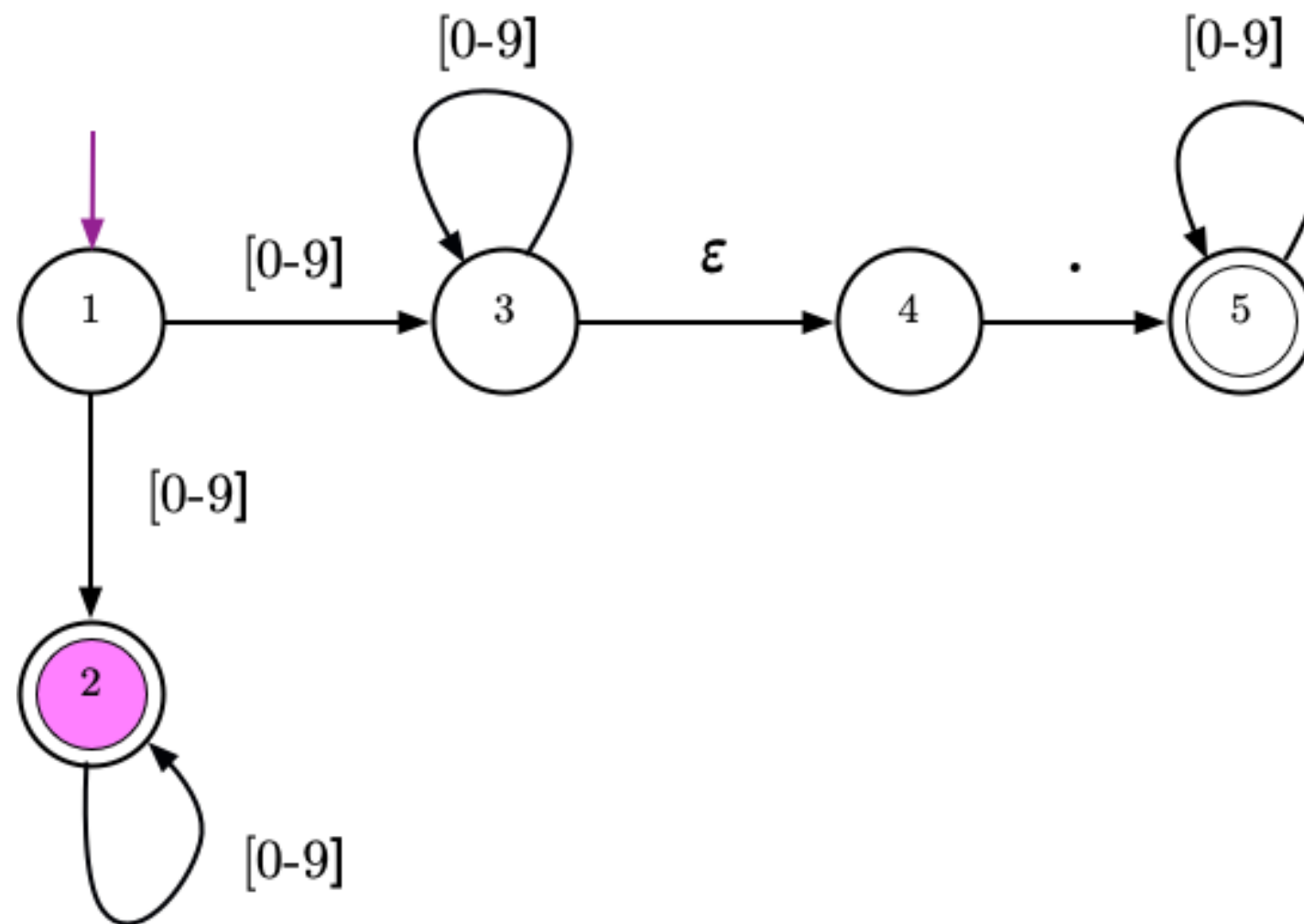
$$[0-9][0-9]^* | [0-9].[0-9]^*$$



Here we see both forms of nondeterminism!

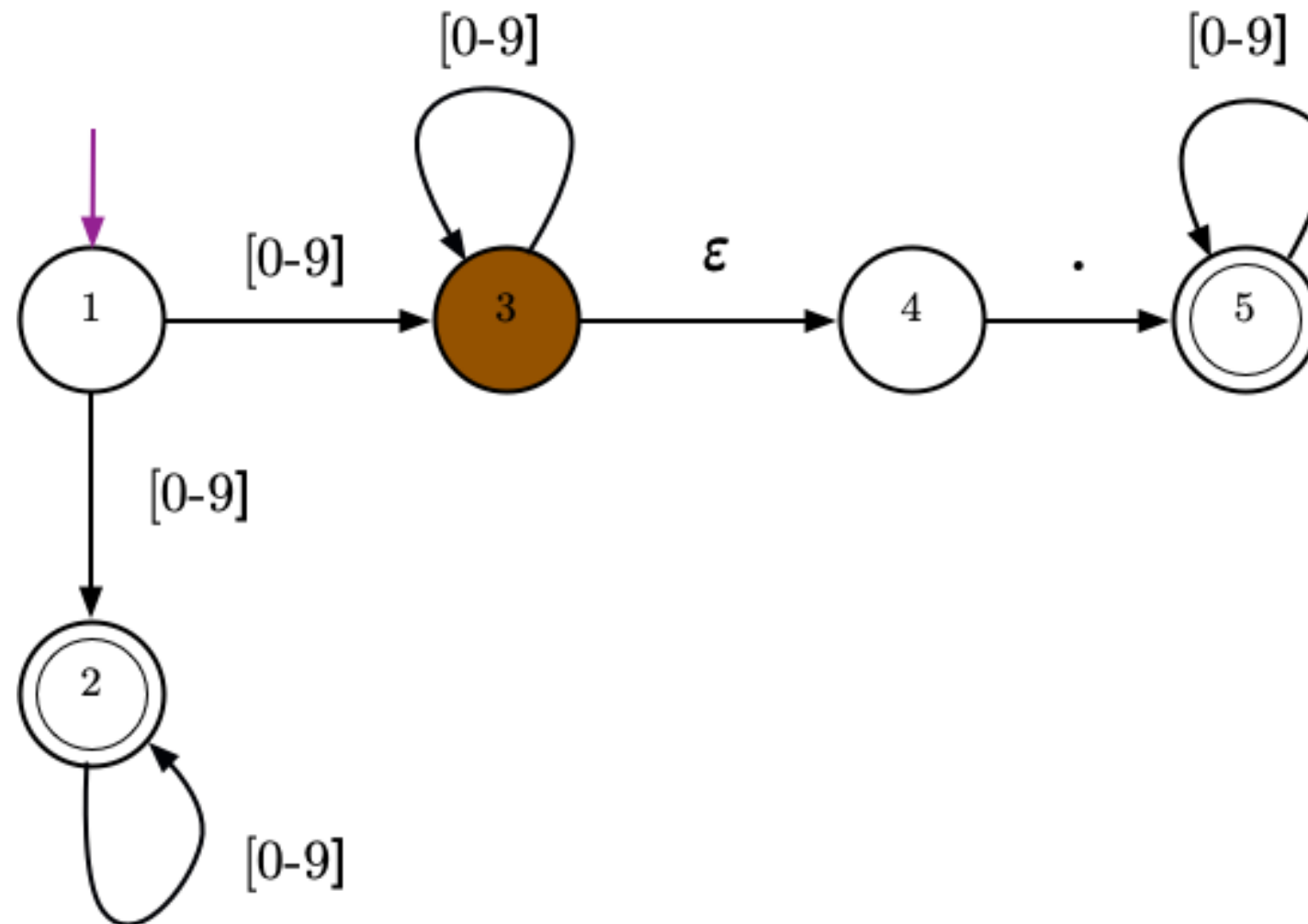
An NFA for decimal numbers

If we let this NFA read 123, we can end in a final state.



An NFA for decimal numbers

But we let this NFA read 123, we can also get stuck in a non-accepting state.:



Nondeterministic Finite Automata (NFA)

- The **initial** state is the state in which the NFA starts reading, a.k.a. the **start** state,
- A **final** state is a state in which execution ends if the input should be accepted

The NFA **accepts** an input string if it can make a valid guess – that is, if there is *some* way of reading all input characters starting from the initial state and finishing in a final state.

The language recognized by an NFA N is called $L(N)$ and is

$$\{w \in \Sigma^* \mid N \text{ accepts } w\}$$

The formal definition

Here are the components of a nondeterministic finite automaton.

Definition (NFA)

Let Σ be an alphabet of (input) characters. An NFA N consists of

- An alphabet Σ of input characters,
- A finite set S of states:
- A start (initial) state $s_0 \in S$
- A set of final (accepting) states $F \subseteq S$ and
- A relation $T \subseteq S \times (\Sigma \cup \{\epsilon\}) \times S$ describing state transitions.

A drawing of the transition graph is fine as a definition in itself. Everything in the above can be read from it.

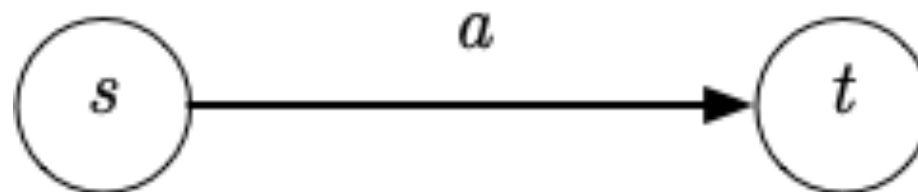
Different texts – different notation

	Sipser	Mogensen	My slides
The set of states	Q	S	S
The start state	q_0	s_0	s_0
Any state	q	s	s
Transitions	A function δ	A relation T or a function move	T
Any transition	$q' \in \delta(q, a)$	$s^a t$	$s \xrightarrow{a} t$

I chose the notation

$$s \xrightarrow{a} t$$

because I think it is easier to read. It is very reminiscent of drawing a transition as



When does an NFA accept its input?

In an NFA a transition $q \xrightarrow{a} q'$ is labelled with either a character $c \in \Sigma$ or ϵ .

- $s_i \xrightarrow{c} s_j \in T$: read character c while in state s_i and go to s_j
- $s_i \xrightarrow{\epsilon} s_j \in T$: if in s_i the NFA can move to s_j without consuming any input; we write $s_i \xrightarrow{\epsilon^*} s_j$ if the NFA can move from s_i to s_j using 0 or more ϵ -transitions.
- Several transitions from a given state (or none!) may exist for some input characters, so T is a relation, not a function.

A string $a_1 \dots a_k$ is accepted by N if we have

$$s_0 \xrightarrow{\epsilon^*} s' \xrightarrow{a_1} s_1 \xrightarrow{\epsilon^*} \dots \xrightarrow{\epsilon^*} s'' \xrightarrow{a_k} \xrightarrow{\epsilon^*} s_f$$

where s_f is some final state. We write $s_0 \xrightarrow{a_1 \dots a_k} s_f$ for such a transition sequence.